Nested loops, `break`, `continue`, `do while`

Sudeep K S, Mahindra University (September 2026)

- Converting 'for' to 'while'; More interesting uses of while

It is easy to see that anything that can be done with a for loop can be done using a while loop as well. We can keep initialization part statement just before the while loop (example: i = 0;), add the update statement (like i++;) inside the block of statements that get executed repeatedly and keep only the condition part at 'while'.

See the below example to understand this better.

```
for (i = 1; i <= 10; i++)
    sum = sum + i;
```

Equivalent code using while loop:

```
i = 1;
while (i <= 10)
{
   sum = sum + I;
   i++;
}
```

However, the other way is not true -- everything that can be done with a while loop may not be easily possible with a 'for' loop. Below is an example for an intelligent use of 'while' -- a program that finds the GCD (greatest common divisor) or two given integers (this was also done in class):

```
int a, b, temp, gcd;

printf("enter the bigger number a\n");
scanf("%d", &a); // reads the number a
printf("enter the other number b\n");
scanf("%d", &b); // reads the number b, such that b <= a

while (b != 0) {
    temp = b;
    b = a % b; // second number now becomes a mod b
    a = temp; // first number gets old value of b
}

gcd = a; // when second number is 0, gcd is a itself
```

Now, we can print the GCD with a `printf` statement as follows:

```
printf(“GCD of %d and %d is %d”, a, b, gcd);
```

- Nested loops: One or more loop(s) inside another loop

Example: A simple double nested for loop: What does it print?

```
for (i = 1; i <= 10; i++)
   for (j = 1; j <= 20; j++)
   {
       printf("i is %d\n", i);
       printf("j is %d\n", j);
       printf("%d x %d is %d\n", i, j, i * j);
   }
```

Note that there are no curly braces used for the outer for loop here, as we have only one statement (one for loop) in the outer ‘for’ loop’s statement block. For the inner for loop, the block contains three statements; so they are grouped together using curly brackets.

- **`break`** and **`continue`** statements – breaking out of loops

We already used ‘`break`’ statement in an example we saw for the for loop, while checking if a given number is prime or not. Let us take a closer look at that part of the code:

```
flag = false;
for (i = 2; i < n; i++)
{
    if (n % i == 0) {
        flag = true;
        printf("n is COMPOSITE and %d is a factor", i);
        break;
    }
}
```

Here, when it reaches ‘`break`’, the execution breaks out of the for loop. It does not check for the remaining values of I, even if the for loop condition holds. Because we have already found a factor of the given number, so we know it cannot be prime, it is a composite number.

Below is an example for use of 'break' statement in a while loop.

```
int a;
printf("enter numbers, end with -1\n");
while (true)
{
    scanf("%d", &a); // reads the number a
    if (a == -1)
        break;
    printf("the number entered is %d", a); // prints the number a
}
```

This will keep reading numbers as long as the number entered is not '-1' and print them one after the other. Note that the condition given in the while loop here just says 'true', which means it goes on forever until the program comes out of it with the help of this break statement.

Use of 'true' requires #include<stdbool.h> in the beginning. Otherwise it gives an error.

Similar to break statement is a continue statement, which **skips the remaining part within the loop block of statements** and goes for the next cycle of the loop. Below is an example.

```
sum = 0;
for (i = 1; i <= 10; i++)
{
       if (i % 2 == 0 || i % 5 == 0) continue;
       sum = sum + i;
}
```

This skips the line that updates the sum when i is a multiple of 2 or 5 (i mod 2 is 0 OR i mod 5 is 0); so 'sum' becomes 1 + 3 + 7 + 9 = 20 at the end of it. Verify it yourself.

- **do while**: A variant of while loop, with condition check at the end

A different kind of while loop first lists the statements and puts the condition at the end. Below is the syntax:

```
do
{
    // sequence of statements
}
while (condition);
```

Example:

```
int a, b;
a = 7; b = 3;

do
{
   a = a + 2;
   printf("inside while: a is %d and b is %d\n", a, b);
}

while (a <= b);
```

Here, the only difference is that the sequence of statements inside the body of this loop get executed at least once. The first condition check happens after one cycle of execution of that block. Try running to verify it yourself.